gRPC & Protocol Buffers

.proto contracts, HTTP/2, deadlines and safe evolution

Watch the 4-min video

In one line: gRPC lets services call each other through code generated from one `.proto` contract, sending compact binary protobuf over HTTP/2 with deadlines and standard status codes; field numbers, not names, are the identity you must never break.

Your orders service is written in Python. The inventory service is written in Go. They call each other **thousands of times per second**. Hand-written JSON clients drift apart (one side renames a field, the other breaks at runtime), and every call pays for verbose text. gRPC with protocol buffers fixes both problems.

1 One contract: the .proto file

You describe the API once, in a `.proto` file, using an **interface definition language** (IDL). It defines a **service** with RPC methods, and **messages** with typed fields. Each field has a **field number**.

```
syntax = "proto3";
package inventory.v1;

service Inventory {
  rpc CheckStock(CheckStockRequest) returns (CheckStockReply);
}
message CheckStockRequest { string sku = 1;  int32 quantity = 2; }
message CheckStockReply   { bool available = 1;  int32 in_stock = 2; }
```

The **protoc** compiler generates client and server code for Python, Go, Java and more. Both sides share one contract, **checked at build time**, not discovered in production.

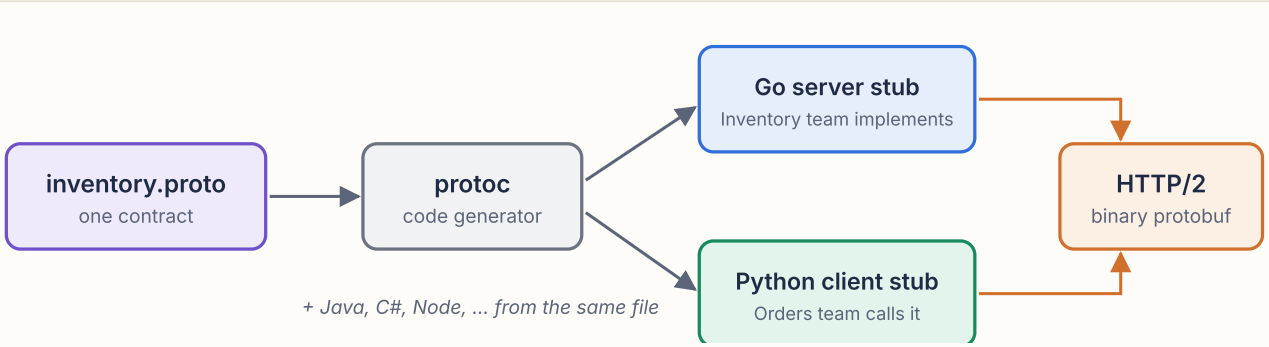


Figure 1. One .proto file produces typed code for every language. Neither team writes serialization code.

2 Compact on the wire

Protobuf is a **binary** format. JSON repeats field names in every message; protobuf tags each field with its small number. Integers use **variable-length encoding** (varint): 3 takes one byte, 300 takes two. Messages are often several times smaller than JSON, and much faster to parse because there is no text to scan.



Figure 2. The same request. Each protobuf tag byte packs the field number and wire type: 0x0A = field 1, length-delimited; 0x10 = field 2, varint.

3 HTTP/2, call types, deadlines and status codes

gRPC runs over **HTTP/2**, so many calls share one TCP connection as **multiplexed streams**: no connection per call, and one slow call does not block the others. There are four call types:

Type	Shape	Example
Unary	one request → one response	`CheckStock`
Server streaming	one request → stream of responses	watch stock changes for a SKU
Client streaming	stream of requests → one response	upload a batch of stock counts
Bidirectional	both sides stream at once	live warehouse sync

Every call should carry a deadline: the latest time the caller will wait. It **propagates** across services, so downstream work stops once nobody is waiting for the answer. Errors use a standard set of **status codes**, such as `NOT_FOUND`, `DEADLINE_EXCEEDED` and `UNAVAILABLE` (usually safe to retry with backoff).

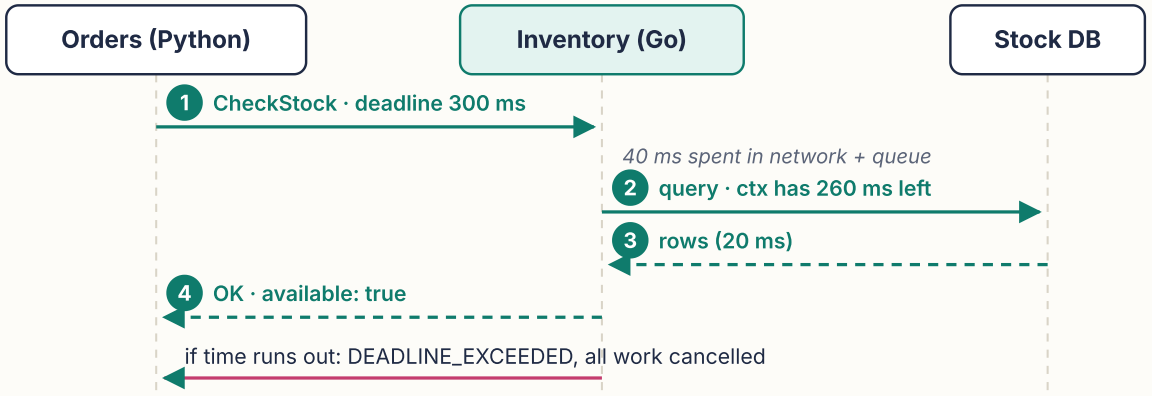


Figure 3. The 300 ms budget travels with the call. Inventory knows it has 260 ms left and passes that on; if time runs out, every hop stops instead of doing useless work.

WORKED EXAMPLE: CHECKSTOCK END TO END

The Go team implements the generated server interface; the Python orders service calls the generated stub with a 300 ms deadline. Neither side writes serialization code.

```
# Python (orders)
stub = inventory_pb2_grpc.InventoryStub(channel)
reply = stub.CheckStock(
    inventory_pb2.CheckStockRequest(sku="A-1001", quantity=3), timeout=0.3)
// Go (inventory)
func (s *server) CheckStock(ctx context.Context, r *pb.CheckStockRequest)
(*pb.CheckStockReply, error) {
    n, err := s.db.Stock(ctx, r.Sku)   // ctx carries the remaining deadline
    return &pb.CheckStockReply{Available: n >= r.Quantity, InStock: n}, err
}
```

4 Evolving a protobuf schema safely

Field numbers are the real identity on the wire, not names. A renamed field still works; a renumbered one silently breaks every old client.

- **Add** fields with **new numbers**. Safe: old readers simply skip unknown fields.
- **Never change** a field's number or type.
- **When you remove a field, reserve its number** (and name), so nobody reuses it with a different meaning.
- **Missing fields read as default values** (0, empty string, false). Design for that: a missing `quantity` looks like 0. Use `optional` or a wrapper when “not set” must differ from zero.

```
message CheckStockRequest {
  reserved 3;  reserved "warehouse";   // removed in v1.4, never reuse
  string sku = 1;
  int32 quantity = 2;
  string region = 4;                  // new, old servers ignore it
}
```

5 gRPC inside, REST outside

Browsers cannot fully control HTTP/2 frames and trailers, which gRPC relies on. Web apps need **gRPC-Web**, with a proxy such as **Envoy** translating. That is one reason for the common pattern: **gRPC internally, REST externally**. Inside, you control both sides and want speed, strict contracts, streaming and deadlines. Outside, partners expect plain HTTP and JSON that works with curl, browsers and every language.

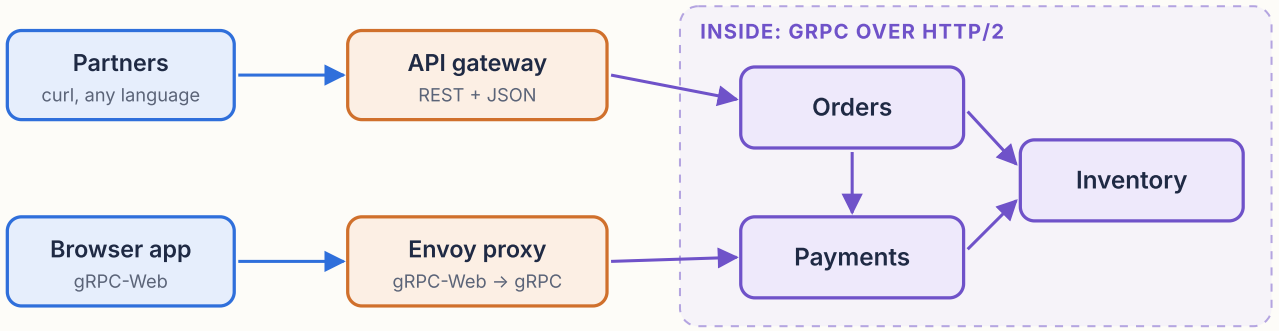


Figure 4. External traffic enters as REST or gRPC-Web; everything behind the edge speaks gRPC.

COMMON MISTAKES

- Calls without deadlines: one slow dependency ties up threads across the whole call chain.
- Reusing or renumbering a field number, so old clients decode garbage without any error.
- Treating 0 or "" as “the client sent this value” when it may simply be missing.
- Opening a new channel per call instead of reusing one, losing HTTP/2 multiplexing.

6 Key terms

Protobuf: binary format defined in .proto

protoc: compiler that generates stubs

Field number: a field's identity on the wire

Deadline: absolute time limit, propagated

reserved: blocks reuse of removed numbers

gRPC-Web: browser variant via a proxy

7 Interview questions

Q1. Why is protobuf smaller and faster than JSON? BASIC

It sends a 1-byte tag (field number + type) instead of the field name, and varint integers instead of digits as text. {"sku":"A-1001","quantity":3} is 29 bytes in JSON and 10 in protobuf, and parsing needs no text scanning.

Q2. Why use gRPC internally and REST externally? SCENARIO

Inside you control both ends: compact binary, HTTP/2 multiplexing, generated clients, streaming and deadlines pay off at thousands of calls per second. Outside, REST with JSON is universal, human-readable, cache-friendly and works in any browser and with curl.

Q3. How do you evolve a protobuf schema safely? SCENARIO

Add fields with new numbers; never change or reuse existing numbers or types; mark deleted numbers (and names) as reserved; and remember missing fields read as defaults, so design for 0 and "".

Q4. Why should every gRPC call have a deadline, and why must it propagate? TRICKY

Without one, a stuck dependency holds the caller forever and threads pile up. If Orders gives up after 300 ms but Inventory keeps querying for 2 s, that work is wasted load. Propagation passes the remaining 260 ms on, so every hop stops together with DEADLINE_EXCEEDED.

Q5. Someone renamed field quantity to qty and kept number 2. Did anything break? TRICKY

Not on the wire: the bytes carry only number 2, so old and new binaries still talk; only code (and JSON mappings) using the name change. Changing the number to 5 would silently break every old client.

Next: SD-C-004 · REST vs GraphQL vs gRPC backendroadmap.com